

1. Explain different SQL data types with suitable examples (10 Points)

SQL data types define what kind of data we can store in a table.

1. **CHAR(n)** – Fixed length text. Always stores exact number of characters. Example: `CHAR(5)` → "HELLO"
2. **VARCHAR(n)** – Variable length text. Stores only required characters. Example: `VARCHAR(20)` → "Tanmay"
3. **INT** – Used to store whole numbers. Example: `INT` → 100
4. **SMALLINT** – Used for small range integers. Example: `SMALLINT` → 10
5. **NUMERIC(p,d)** – Used for exact decimal values (p = total digits, d = digits after decimal). Example: `NUMERIC(5,2)` → 123.45
6. **REAL** – Used for approximate decimal values. Example: `REAL` → 45.67
7. **DOUBLE PRECISION** – Stores large decimal values with more accuracy. Example: `DOUBLE` → 12345.6789
8. **FLOAT(n)** – Floating point number with precision. Example: `FLOAT(10)` → 123.456
9. **DATE** – Stores date values. Example: `DATE` → '2026-04-24'
10. **TIME** – Stores time values. Example: `TIME` → '10:30:00'

2. Differentiate between DDL, DML, DCL, and TCL commands (10 Points)

1. **DDL (Data Definition Language)** – Used to define database structure. Example:

CREATE , ALTER , DROP

2. DDL commands work on tables and schema (structure of database).
3. DDL changes are **permanent** (auto-commit).
4. **DML (Data Manipulation Language)** – Used to modify data in tables. Example:

INSERT , UPDATE , DELETE

5. DML works on actual data (records/rows).
6. DML changes can be **rolled back**.

7. **DCL (Data Control Language)** – Used to control access/permissions. Example: GRANT ,

REVOKE

8. DCL controls user rights and security.
9. **TCL (Transaction Control Language)** – Used to manage transactions. Example:

COMMIT , ROLLBACK , SAVEPOINT



10. TCL ensures data consistency and controls transaction execution.

3. Write SQL commands to Create, Alter and Drop table (10 Points)

1. **CREATE TABLE** – Used to create a new table in database.
2. **Syntax:** `CREATE TABLE table_name (column_name datatype, ...);`
3. **Example:** `CREATE TABLE student (ID INT, Name VARCHAR(20));`
4. **ALTER TABLE** – Used to modify existing table structure.
5. We can add or remove columns using ALTER.
6. **Syntax:** `ALTER TABLE table_name ADD column_name datatype;`
7. **Example:** `ALTER TABLE student ADD Age INT;`
8. **DROP TABLE** – Used to delete the entire table.
9. **Syntax:** `DROP TABLE table_name;`
10. **Example:** `DROP TABLE student;`

4. Write SQL queries for Insert, Update, Delete (10 Points)

1. **INSERT** – Used to add new data into a table.
2. **Syntax:** `INSERT INTO table_name VALUES (value1, value2, ...);`
3. **Example:** `INSERT INTO student VALUES (1, 'Tanmay');`
4. **UPDATE** – Used to modify existing data in a table.
5. **Syntax:** `UPDATE table_name SET column_name = value WHERE condition;`
6. **Example:** `UPDATE student SET Name = 'Rahul' WHERE ID = 1;`
7. **DELETE** – Used to remove data from a table.
8. **Syntax:** `DELETE FROM table_name WHERE condition;`
9. **Example:** `DELETE FROM student WHERE ID = 1;`
10. These commands help in managing and modifying table data efficiently.

5. Explain SELECT statement with WHERE, ORDER BY, GROUP BY, HAVING (10 Points)

1. **SELECT** – Used to retrieve data from a table.
2. **Basic Syntax:** `SELECT column_name FROM table_name;`
3. **Example:** `SELECT Name FROM student;`
4. **WHERE** – Used to filter records based on condition.
5. **Example:** `SELECT * FROM student WHERE ID = 1;`
6. **ORDER BY** – Used to sort data (ASC/DESC).
7. **Example:** `SELECT * FROM student ORDER BY Name ASC;`
8. **GROUP BY** – Used to group rows with same values (with functions like COUNT, SUM).
9. **Example:** `SELECT dept, COUNT(*) FROM student GROUP BY dept;`
10. **HAVING** – Used to filter grouped data (after GROUP BY). Example: `SELECT dept, COUNT(*) FROM student GROUP BY dept HAVING COUNT(*) > 2;`

6. Types of SQL Joins (10 Points)

1. **JOIN** – Used to combine data from two tables based on a common column.
2. **INNER JOIN** – Returns only matching records from both tables.
3. **Example:** `SELECT * FROM A INNER JOIN B ON A.id = B.id;`
4. **LEFT JOIN (LEFT OUTER JOIN)** – Returns all records from left table and matching from right.
5. If no match, NULL values are shown for right table.
6. **Example:** `SELECT * FROM A LEFT JOIN B ON A.id = B.id;`
7. **RIGHT JOIN (RIGHT OUTER JOIN)** – Returns all records from right table and matching from left.
8. If no match, NULL values are shown for left table.
9. **Example:** `SELECT * FROM A RIGHT JOIN B ON A.id = B.id;`
10. **FULL JOIN (FULL OUTER JOIN)** – Returns all records from both tables (matched + unmatched). **Example:** `SELECT * FROM A FULL JOIN B ON A.id = B.id;`

6. Types of SQL Joins (10 Points)

1. **JOIN** – Used to combine data from two tables based on a common column.
2. **INNER JOIN** – Returns only matching records from both tables.
3. **Example:** `SELECT * FROM A INNER JOIN B ON A.id = B.id;`
4. **LEFT JOIN (LEFT OUTER JOIN)** – Returns all records from left table and matching from right.
5. If no match, NULL values are shown for right table.
6. **Example:** `SELECT * FROM A LEFT JOIN B ON A.id = B.id;`
7. **RIGHT JOIN (RIGHT OUTER JOIN)** – Returns all records from right table and matching from left.
8. If no match, NULL values are shown for left table.
9. **Example:** `SELECT * FROM A RIGHT JOIN B ON A.id = B.id;`
10. **FULL JOIN (FULL OUTER JOIN)** – Returns all records from both tables (matched + unmatched). **Example:** `SELECT * FROM A FULL JOIN B ON A.id = B.id;`

7. Views in SQL (10 Points)

1. **View** – A virtual table based on a SQL query (does not store data physically).
2. It shows data from one or more tables.
3. **CREATE VIEW** – Used to create a view.
4. **Syntax:** `CREATE VIEW view_name AS SELECT column_name FROM table_name;`
5. **Example:** `CREATE VIEW student_view AS SELECT Name FROM student;`
6. **UPDATE VIEW** – Used to modify data through view (if allowed).
7. **Syntax:** `UPDATE view_name SET column = value WHERE condition;`
8. **Example:** `UPDATE student_view SET Name = 'Rahul' WHERE Name = 'Tanmay';`
9. **DROP VIEW** – Used to delete a view.
10. **Syntax & Example:** `DROP VIEW student_view;`

8. Aggregation Functions in SQL (10 Points)

1. **Aggregation functions** are used to perform calculations on a group of values and return a single result.
2. **COUNT()** – Counts number of rows.
3. **Example:** `SELECT COUNT(*) FROM student;`
4. **SUM()** – Calculates total of a column.
5. **Example:** `SELECT SUM(marks) FROM student;`
6. **AVG()** – Finds average value.
7. **Example:** `SELECT AVG(marks) FROM student;`
8. **MAX()** – Finds highest value.
9. **MIN()** – Finds lowest value.
10. **Example:** `SELECT MAX(marks), MIN(marks) FROM student;`

9. Set Operations in SQL (10 Points)

1. **Set Operations** – Used to combine results of two or more SELECT queries.
2. **UNION** – Combines results and removes duplicate rows.
3. **Example:** `SELECT Name FROM student1 UNION SELECT Name FROM student2;`
4. **INTERSECT** – Returns only common records from both queries.
5. **Example:** `SELECT Name FROM student1 INTERSECT SELECT Name FROM student2;`
6. **MINUS (EXCEPT)** – Returns records from first query that are not in second.
7. **Example:** `SELECT Name FROM student1 MINUS SELECT Name FROM student2;`
8. All queries must have same number of columns and compatible data types.
9. Column names are taken from the first query.
10. These operations help in comparing and combining data from multiple tables.

10. SQL Operators and Predicates (10 Points)

1. SQL Operators – Used to perform operations on data values.
2. Arithmetic Operators – `+` , `-` , `*` , `/`
3. Example: `SELECT salary*12 FROM employee;`
4. Comparison Operators – `=` , `>` , `<` , `>=` , `<=` , `<>`
5. Example: `SELECT * FROM student WHERE marks > 50;`
6. Logical Operators – `AND` , `OR` , `NOT`
7. Example: `SELECT * FROM student WHERE marks > 50 AND dept = 'CS';`
8. Predicates – Special conditions used in WHERE clause.
9. Examples: `BETWEEN` , `LIKE` , `IN` , `IS NULL`
10. Example: `SELECT * FROM student WHERE marks BETWEEN 40 AND 80;`

11. Nested Queries (10 Points)

1. **Nested Query (Subquery)** – A query written inside another SQL query.
2. It is usually used inside `WHERE` , `FROM` , or `SELECT` clause.
3. **Single-row subquery** – Returns only one value.
4. Uses operators like `=` , `>` , `<`
5. **Example:** `SELECT * FROM student WHERE marks > (SELECT AVG(marks) FROM student);`
6. **Multi-row subquery** – Returns multiple values.
7. Uses operators like `IN` , `ANY` , `ALL`
8. **Example:** `SELECT * FROM student WHERE dept IN (SELECT dept FROM department);`
9. Nested queries help to perform complex conditions easily.
10. They make queries more powerful and flexible.

12. TCL Commands (10 Points)

1. **TCL (Transaction Control Language)** – Used to manage transactions in database.
2. **COMMIT** – Saves all changes permanently.
3. Example: `COMMIT;`
4. **ROLLBACK** – Undoes changes and returns to last committed state.
5. Example: `ROLLBACK;`
6. **SAVEPOINT** – Creates a point within a transaction.
7. We can rollback to that specific point.
8. Example: `SAVEPOINT sp1;`
9. Rollback to savepoint: `ROLLBACK TO sp1;`
10. TCL ensures data consistency and safe transaction control.

13. DCL Commands (10 Points)

1. **DCL (Data Control Language)** – Used to control access and permissions in database.
2. **CREATE USER** – Used to create a new user.
3. **Syntax:** `CREATE USER username IDENTIFIED BY password;`
4. **Example:** `CREATE USER tanmay IDENTIFIED BY 1234;`
5. **GRANT** – Used to give permissions to a user.
6. **Syntax:** `GRANT privilege ON table_name TO username;`
7. **Example:** `GRANT SELECT, INSERT ON student TO tanmay;`
8. **REVOKE** – Used to remove permissions from a user.
9. **Syntax:** `REVOKE privilege ON table_name FROM username;`
10. **Example:** `REVOKE INSERT ON student FROM tanmay;`

◆ 1. What is PL/SQL?

- 👉 PL/SQL (Procedural Language/SQL) is an extension of SQL
- 👉 It allows you to write programs inside the database

◆ Why needed?

SQL alone:

- Only works on data (SELECT, INSERT, etc.)

PL/SQL:

- Adds logic like:
 - variables
 - loops
 - conditions (IF)
 - functions & procedures

◆ 2. Structure of PL/SQL Block

A PL/SQL program is written in blocks

◆ Basic Structure:

```
SQL  
  
DECLARE  
    -- variables  
BEGIN  
    -- execution statements  
END;
```

◆ 3 Parts Explained

◆ 1. DECLARE (Optional)

👉 Used to declare variables

```
SQL  
  
DECLARE  
    x NUMBER;  
    name VARCHAR2(20);
```



♦ 2. BEGIN (Compulsory)

 Try Codex x

👉 Main execution part

↗ SQL

```
BEGIN  
  x := 10;
```

♦ 3. END (Compulsory)

👉 Marks end of block

↗ SQL

```
END;
```

♦ Complete Example

↗ SQL

```
DECLARE  
  x NUMBER;  
BEGIN  
  x := 10;  
  DBMS_OUTPUT.PUT_LINE(x);  
END;
```



◆ 4. Stored Procedure

👉 A stored procedure is a precompiled program stored in database

◆ Why use?

- Reusable
- Faster execution
- Reduces code repetition

◆ Syntax:

↳ SQL

```
CREATE PROCEDURE procedure_name AS
BEGIN
    -- logic
END;
```

◆ Example:

SQL

```
CREATE PROCEDURE show_msg AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello Tanmay');
END;
```

◆ Call Procedure:

SQL

```
BEGIN
    show_msg;
END;
```

👉 Output: Hello Tanmay

◆ Example with parameter

SQL

```
CREATE PROCEDURE square_num(n NUMBER) AS
BEGIN
    DBMS_OUTPUT.PUT_LINE(n*n);
END;
```



Call:

 Try Codex 

 SQL

```
BEGIN
    square_num(5);
END;
```

 Output: 25

5. Function

 A function is similar to procedure but returns a value

Syntax:

 SQL

```
CREATE FUNCTION function_name
RETURN datatype AS
BEGIN
    RETURN value;
END;
```

◆ Example:

SQL

```
CREATE FUNCTION get_square(n NUMBER)
RETURN NUMBER AS
BEGIN
    RETURN n*n;
END;
```

◆ Calling Function:

SQL

```
SELECT get_square(4) FROM dual;
```

👉 Output: 16

🔥 Difference: Procedure vs Function

Feature	Procedure	Function
Return value	✗ No	✓ Yes
Use in SELECT	✗ No	✓ Yes



Difference: Procedure vs Function

Feature	Procedure	Function
Return value	No	Yes
Use in SELECT	No	Yes
Purpose	Perform task	Compute value

◆ 1. Trigger

👉 A trigger is a block of code that automatically runs when an event occurs in a table.

◆ Events:

- INSERT
- UPDATE
- DELETE

◆ Why use triggers?

- Automatic actions
- Data validation
- Auditing (tracking changes)

◆ Syntax:

SQL

```
CREATE TRIGGER trigger_name  
BEFORE/AFTER INSERT/UPDATE/DELETE  
ON table_name  
BEGIN  
    -- logic  
END;
```



◆ 2. Assertion

👉 An assertion is a condition that must always be true in the database.

◆ Purpose:

- Maintain data integrity
 - Prevent invalid data
-

◆ Syntax:

📄 SQL

```
CREATE ASSERTION assertion_name  
CHECK (condition);
```

◆ Example:

📄 SQL

```
CREATE ASSERTION check_salary  
CHECK (salary > 0);
```

👉 Meaning:

- Salary can never be negative



⚠ Note:

- 👉 Assertions are not supported in many databases (like MySQL)
 - 👉 Instead, we use constraints (CHECK)
-

◆ 3. Role

👉 A role is a collection of permissions (privileges)

◆ Why use roles?

- Manage many users easily
 - Assign permissions in group
-

◆ Example:

↗ SQL

```
CREATE ROLE student_role;
```

↗ SQL

```
GRANT SELECT, INSERT ON student TO student_role;
```

↗ SQL

```
GRANT student role TO user1;
```



👉 Meaning:

- user1 gets all permissions of student_role
-

◆ Real-life example:

👉 Like a job role in company

- "Manager" role → has many permissions
-

◆ 4. Privilege

👉 A privilege is a permission to perform actions on database

◆ Types:

- `SELECT` → read data
 - `INSERT` → add data
 - `UPDATE` → modify
 - `DELETE` → remove
-

◆ Example:

🔗 SQL

```
GRANT SELECT ON student TO user1;
```



👉 user1 can only view data

◆ Revoke privilege:

↳ SQL

```
REVOKE SELECT ON student FROM user1;
```


Q3(a)

↪ SQL

```
SELECT d.Department_Name, AVG(e.Salary)
FROM Employee e
JOIN Department d
ON e.Department_ID = d.Department_ID
GROUP BY d.Department_Name
HAVING AVG(e.Salary) > 30000;
```

Q3(b)

↪ SQL

```
CREATE OR REPLACE PROCEDURE student_result(s_id NUMBER) AS
    total NUMBER;
    percentage NUMBER;
BEGIN
    SELECT SUM(Marks)
    INTO total
    FROM Student_Marks
    WHERE Student_ID = s_id;

    percentage := total / 5;

    DBMS_OUTPUT.PUT_LINE('Total Marks: ' || total);
    DBMS_OUTPUT.PUT_LINE('Percentage: ' || percentage);
END;
```



Q4(a)

1.

`</>` SQL

```
SELECT * FROM Employee  
WHERE Department = 'IT';
```

2.

`</>` SQL

```
SELECT * FROM Employee  
WHERE Salary BETWEEN 40000 AND 80000;
```

3.

`</>` SQL

```
SELECT * FROM Employee  
WHERE Name LIKE 'S%';
```

4.

`</>` SQL

```
SELECT * FROM Employee  
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```





Q4(b)

SQL

```
CREATE OR REPLACE TRIGGER audit_balance
AFTER UPDATE OF balance ON Account
FOR EACH ROW
BEGIN
    INSERT INTO Audit_Log (old_balance, new_balance)
    VALUES (:OLD.balance, :NEW.balance);
END;
```

SQL



```
SELECT dep.dep_name, AVG(emp.salary)
FROM employee emp
JOIN dep
ON emp.dep_id = dep.dep_id
GROUP BY dep.dep_name
HAVING AVG(emp.salary) > 30000;
```

employee

emp_id	name	salary	dep_id
1	Ram	40000	101
2	Shyam	35000	101
3	Amit	25000	102
4	Neha	50000	103

dep

dep_id	dep_name
101	HR
102	Sales
103	IT

dep_name	AVG(emp.salary)
-----------------	------------------------

HR	37500
----	-------

IT	50000
----	-------